

Product Requirements Document (PRD)

ProofGuard — Secure Project Proofing & Watermark-Gated Delivery Platform

Version: 1.0 Author: Raymond (Ray) Date: July 2026

1. Overview

1.1 Problem Statement

Graphic designers routinely lose income because clients preview finished work, then decline to pay — since the design was already viewable and, in many cases, downloadable in usable quality. There is no low-cost, self-hosted way for independent designers to share previews safely without expensive third-party proofing tools.

1.2 Solution

ProofGuard lets a designer upload a finished project and generate a shareable client-facing link. The client can view (and optionally download) a protected version of the file — visibly watermarked and degraded — until payment is completed. Once paid, the client unlocks the clean, original, full-resolution file.

1.3 Goals

- Give designers a way to share proofs without risking unpaid usage
- Make the "pay to unlock" moment frictionless for the client
- Keep infrastructure cost low and predictable (aligned with Ray's existing cost-conscious Cloudflare-based architecture)
- Ship an MVP that is realistic to build and maintain as a solo/small-team project

1.4 Non-Goals (MVP)

- No built-in contract/e-signature system (future scope)
 - No multi-currency invoicing/tax handling beyond basic payment capture
 - No native mobile app (web-responsive only for MVP)
 - No team/agency multi-seat accounts (single designer accounts only for MVP)
-

2. Target Users

Persona	Description	Needs
Designer (primary user)	Freelance or small-studio graphic designer	Upload work, generate protected client link, get paid before releasing original
Client (secondary user, no account)	Business or individual receiving design work	View work clearly enough to evaluate it, pay easily, receive original after payment

3. Core User Flows

3.1 Designer Flow

1. Designer signs up / logs in (Firebase Auth)
2. Uploads a project file (image/PDF) + sets title, client email, price
3. System auto-generates a watermarked preview version and stores both original (R2, private) and preview (R2, public-readable via signed URL)
4. Designer copies the generated client link and sends it manually (email/WhatsApp/etc.) — no in-app messaging in MVP
5. Designer sees status updates: Not Viewed → Viewed → Paid
6. Once paid, designer sees it reflected on the dashboard and in analytics

3.2 Client Flow

1. Client opens the link (no login required)
 2. Sees watermarked preview in-browser (protected against right-click/drag-save where feasible)
 3. Client can attempt "Download" → receives a heavily watermarked, degraded-quality file (safe to send, unusable for real work)
 4. Client clicks "Pay to Unlock Original"
 5. Completes payment
 6. Immediately receives access to the clean original file via a time-limited signed download link
-

4. Feature Requirements

4.1 MVP Features (Must-Have)

- Designer authentication (sign up, log in, log out) — Firebase Auth
- Project upload (image/PDF) with title, client email, price
- Automatic watermark generation (server-side, not removable via browser inspection)
- Public client preview page (unique link per project, no login)
- "Download watermarked preview" option
- Payment integration (Stripe or Paystack, given Ghana context — see Open Questions)
- Post-payment unlock: signed, time-limited download URL to the original file
- Designer dashboard: list of projects + status (Not Viewed / Viewed / Paid)
- Basic analytics: total projects, total viewed, conversion rate, total earnings

4.2 Nice-to-Have (Post-MVP)

- Watermark intensity toggle (Light / Medium / Heavy)
- Email notifications (client viewed / client paid) — via a lightweight email API
- Expiring links (proof link dies after X days if unpaid)
- Multiple files per project (batch proofing)
- Designer custom branding on the client preview page

5. System Architecture

5.1 Stack Summary

Layer	Technology	Purpose
Frontend	Next.js	Designer dashboard + public client preview pages
Backend / Core Logic	C# (ASP.NET Core Web API)	Business logic, watermarking pipeline, payment webhook handling, API endpoints
Authentication	Firebase Auth	Designer sign-up/login only (ID tokens verified server-side via JWKS — same pattern as TrustLink Africa)
Database	Cloudflare D1	Stores project metadata, client info, status, payment records

Layer	Technology	Purpose
File Storage	Cloudflare R2	Stores original files (private) and generated watermarked previews (public/signed access)
Payments	Stripe or Pystack (TBD — see Open Questions)	Payment capture + webhook for unlock trigger

5.2 High-Level Architecture Flow

1. **Next.js frontend** calls the **ASP.NET Core API** for all designer actions (upload, list projects, view analytics) and for the public client preview data.
2. **ASP.NET Core backend** validates Firebase ID tokens for designer-only routes (JWKS verification, no Firebase Admin SDK dependency needed).
3. On upload, the backend:
 - Stores the original file in **R2** (private bucket)
 - Runs the watermarking process (server-side image processing) to generate a preview version
 - Stores the watermarked preview in **R2** (public-readable bucket or signed URL access)
 - Writes project metadata to **D1**
4. Public client preview page fetches watermarked preview via signed R2 URL — original file is never exposed to the client until payment.
5. On successful payment webhook, backend updates **D1** status to "Paid" and generates a **time-limited signed URL** to the original file in R2 for the client to download.

5.3 Why This Stack Fits

- Mirrors Ray's existing TrustLink Africa pattern: Firebase for auth only, Cloudflare D1/R2 for everything else — keeps costs predictable and avoids Firestore/Storage pricing unpredictability.
- C# backend keeps core logic (watermarking, payment verification, access control) centralized and testable, separate from the frontend.
- Next.js is well-suited for a mixed public (client preview) + authenticated (designer dashboard) app, with good SEO/shareability for client links.

6. Data Model (Cloudflare D1)

Field	Type	Notes
id	TEXT (PK)	Firebase UID
name	TEXT	
email	TEXT	
created_at	DATETIME	

Projects

Field	Type	Notes
id	TEXT (PK)	UUID
designer_id	TEXT (FK)	
title	TEXT	
client_email	TEXT	
price	DECIMAL	
status	TEXT	Not Viewed / Viewed / Paid
original_file_key	TEXT	R2 object key (private)
preview_file_key	TEXT	R2 object key (watermarked)
public_link_token	TEXT	Unique token for client URL
created_at	DATETIME	
viewed_at	DATETIME (nullable)	
paid_at	DATETIME (nullable)	

Payments

Field	Type	Notes
id	TEXT (PK)	

Field	Type	Notes
project_id	TEXT (FK)	
amount	DECIMAL	
payment_provider_ref	TEXT	Stripe/Paystack transaction ID
status	TEXT	Pending / Completed / Failed
created_at	DATETIME	

7. API Endpoints (ASP.NET Core)

Designer (Authenticated — Firebase token required)

- `POST /api/projects` — create/upload a new project
- `GET /api/projects` — list designer's projects
- `GET /api/projects/{id}` — get single project details
- `GET /api/analytics` — designer's stats summary

Public (No auth — token-based access via link)

- `GET /api/public/projects/{publicLinkToken}` — fetch project preview data
- `POST /api/public/projects/{publicLinkToken}/view` — mark as viewed
- `GET /api/public/projects/{publicLinkToken}/download-preview` — get watermarked file
- `POST /api/public/projects/{publicLinkToken}/pay` — initiate payment
- `POST /api/webhooks/payment` — payment provider webhook → triggers unlock

Unlock (Post-payment, token-based)

- `GET /api/public/projects/{publicLinkToken}/download-original` — signed URL, valid only after `(status = Paid)`, time-limited (e.g., 24 hours)

8. Watermarking Approach

This is the core protective mechanism and needs to be **server-side**, not CSS/client-side overlay (which can be stripped via browser dev tools).

Recommended approach:

- Use a C# image-processing library (e.g., **ImageSharp**) to bake the watermark directly into the pixel data of the preview file before it's ever sent to the client.
- Apply: diagonal repeating text/logo watermark + reduced resolution (e.g., cap preview at 800px wide) + slight noise/grain overlay to discourage clean screenshots.
- Never expose the original file URL to the frontend at any point pre-payment — the API should only ever return the preview file key/URL until `status = Paid`.
- For PDFs, apply the same principle using a PDF library to stamp/degrade each page server-side.

Security note: true protection is server-side generation + never exposing the original — no client-side watermark can fully prevent a determined client from removing it, so the goal is to make removal impractical, not impossible.

9. Non-Functional Requirements

- **Cost efficiency:** stay within Cloudflare D1/R2 free-tier or low-cost usage for MVP scale (aligned with Ray's stated infrastructure priorities)
 - **Security:** signed, time-limited URLs for all file access; no permanent public URLs to original files
 - **Performance:** preview page should load in under 2 seconds on average connection
 - **Reliability:** payment webhook handling must be idempotent (avoid double-processing on retries)
-

10. Open Questions

1. **Payment provider:** Stripe (better docs, but limited direct support in Ghana) vs. Paystack (strong Ghana/Africa support) — needs a decision before building the payment module.
 2. **Watermark removal risk:** how much server-side protection is "enough" for the sem/project scope vs. production-grade — worth scoping down for an academic deadline.
 3. **File types supported in MVP:** images only, or images + PDF from day one?
 4. **Link expiry:** should unpaid preview links expire automatically (e.g., after 14 days)?
-

11. Suggested MVP Build Order

1. Firebase Auth wiring (designer login only)
2. D1 schema + basic CRUD API (Projects, Designers)
3. R2 upload pipeline (original file)
4. Server-side watermarking service (C# + ImageSharp)
5. Public client preview page (Next.js) reading from watermarked R2 file
6. Payment integration + webhook → status update
7. Signed URL unlock flow for original file
8. Designer dashboard + analytics view (Next.js)